

Hardware Design Topics

Sai Mohan Kilambi

April 25, 2026

Contents

1	Introduction	2
2	Register Types	2
2.1	Keeper Register	2
3	Race Conditions	2
3.1	When an update needs to be delayed until current transaction is done	3
4	Common Hardware Blocks	4
4.1	Inferred Latch	4
4.2	Synchronous FIFO	4
4.2.1	Count Logic	4
4.2.2	Write Logic	5
4.2.3	Read Logic	5
4.2.4	Control and Status Logic	6
4.3	Ring Buffer or Queue	6

1 Introduction

This document contains notes on various hardware design topics that I find useful to take note of. We will cover hardware design elements, design patterns, static timing analysis, FPGA topics that are useful and other such things.

2 Register Types

2.1 Keeper Register

A Keeper register latches the value in the current cycle and then holds in until a new update arrives and then it keeps that...so on and so forth. Note that it does not break a timing path.

```
1
2 module keeper_reg #(
3     parameter type T = logic
4 ) (
5     input clk,
6     input reset,
7
8     input logic valid,
9     input T sig_in,
10    output T sig_out
11 );
12
13    logic data;
14    T data_reg;
15
16    assign data = valid ? sig_in : data_reg;
17
18    always_ff @(posedge clk) begin
19        data_reg <= data;
20    end
21
22    assign sig_out = data;
23
24 endmodule
```

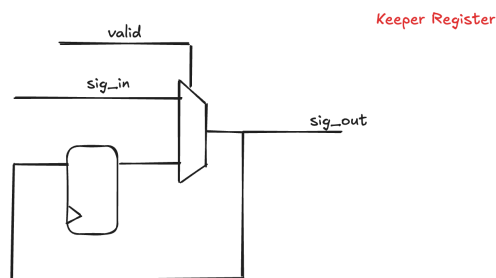


Figure 1: Keeper Reg

3 Race Conditions

This section covers various cases of race conditions and how to handle them.

3.1 When an update needs to be delayed until current transaction is done

Sometimes we have a situation where an update comes in from another block or MMIO and the current block has a state-machine thats operating. If the state machine is in IDLE mode, we want to take the update in immediately. Otherwise we want to keep it pending until the current transaction is complete and then apply it.

Lets say we have a block where

1. An *update* pulse is an input that moves a value from a holding register (that has been programmed via MMIO) into the active register. Lets call these *a_holding_reg* and *a_active_reg*
2. Lets say the block also has a state machine with the following states *IDLE*, *PROCESSING*.

Here are the constraints

1. If in *IDLE* state, then update should reflect immediately.
2. If not in *IDLE* then wait for applying the change.

```
1
2 logic transaction_in_progress;
3
4 // We say transaction is in progress when
5 // 1. State is IDLE and a trigger shows up (this is an input probably into the
   block)
6 // 2. OR, we are in PROCESSING state.
7 assign transaction_in_progress = trigger || state != IDLE;
8
9 /*
10 In this code we track when update needs to be pending.
11 */
12 always_comb begin
13     if ( trigger & transaction_in_progress ) begin
14         transaction_pending_next = 1;
15     end else if ( transaction_pending & !transaction_in_progress ) begin
16         transaction_pending_next = 0;
17     end else
18         transaction_pending_next = transaction_pending;
19 end
20
21 'REG_RC ( transaction_pending, '0, transaction_pending_next )
22
23 // When should we do the update?
24 // 1) When we receive a trigger and we are not in the middle of a transaction
25 // 2) OR, we received a trigger in the middle of a transaction, we waited and now
   we are
26 //     clear to update.
27 assign do_update = (trigger || transaction_pending) & (! transaction_in_progress)
   ;
28
29 // need to hold the data when we receive the trigger in the middle of a
   transaction
30 'REG_EC ( pending_reg, trigger & transaction_in_progress, a_holding_register)
31
32 // What is the source of the update?
33 assign source_data = (trigger & !transaction_in_progress) ? a_holding_reg :
34     (transaction_pending & !transaction_in_progress) ?
35         pending_reg :
36         a_active_reg;
37 'REG_EC ( a_active_reg, do_update, source_data )
```

4 Common Hardware Blocks

4.1 Inferred Latch

There are two kinds of memory elements in hardware design. One is a Flip-Flop and another is a Latch. A flip-flop changes its output state only on a rising or a falling edge of a clock. A latch on the other hand will change its state whenever its input changes state. However when input does not change state it will have to remember its previous output.

In a case or if-else statement that is not complete, a latch may get inferred as shown in ???. This created a combination loop that cannot be timed. This is why inferred latches are harmful for ASIC/FPGA design. In the following code snippet, the `comb_out` variable is not set when `sel` is `2'b11`. This will create a feedback. Always best to have complete case statement list or use *unique case* statement.

```
logic [1:0] sel, comb_out;

always_comb begin
    case(sel)
        2'b00: comb_out = 0;
        2'b01: comb_out = 1;
        2'b10: comb_out = 2;
    endcase
end
```

4.2 Synchronous FIFO

In a Sync FIFO the write and the read operate on the same clock. Let's break this down into its fundamental pieces.

1. Memory unit: This can be a bunch of registers of a certain width or BRAMs in an FPGA.
2. Write Logic: This logic deals with keeping track of the write pointer. The write pointer indicates where the next data element should be written.
3. Read Logic: This portion deals with read pointer. The read pointer tracks the next location to be read.
4. Control and Status Logic: This portion keeps track of status signals such as EMPTY, FULL, ALMOST_EMPTY, ALMOST_FULL etc. COUNT signal will keep track of how many elements are currently present in the FIFO.

We will dig into each of these logic portions individually.

4.2.1 Count Logic

Count keeps track of how many elements are currently in the FIFO. It has the following properties

1. Should increment when there is a write but no read and its not full.
2. Should decrement when there is a read but no write and its not empty.
3. Should stay the same if there is no read OR write or there is a read AND write.

```
always_ff @ (posedge clk) begin
    if ( ~resetn ) begin
        count <= '0;
    end else begin
        count <= count_nxt;
    end
end
```

```

        end
    end

    always_comb begin
        count_nxt = count;

        case ( {we, re} ) begin
            2'b10: count_nxt = count + 1;
            2'b01: count_nxt = count - 1;
        endcase
    end
end

```

4.2.2 Write Logic

We first want to answer the following questions:

1. When should a write occur? It should occur when the FIFO is not FULL. The FULL status signal should be used as a protection circuit for the writes. We can deal with this with valid, ready handshake on the write side. Valid is an input and ready should be high only if there is space in the FIFO.
2. Pointer logic: Write pointer should increment whenever there is a valid write transaction. Remember that it indicates where the next write should happen.

```

// Write enable
assign we = in_valid & in_ready;

// Full Logic
assign full = ~(count < SIZE);

// In Ready Logic
assign in_ready = !full;

// Write pointer
always_ff@ ( posedge clk ) begin
    if (!resetn) begin
        wptr <= '0;
    end else begin
        if ( we )
            wptr <= f_incr_ptr ( wptr, SIZE );
    end
end

function addr_t f_incr_ptr(addr_t ptr, int SIZE);
    addr_t int_ptr;

    int_ptr = ptr < SIZE - 1 ? ptr+1 : 0;

    return int_ptr;
endfunction;

```

4.2.3 Read Logic

We want to read when the down stream logic is ready to pop and when FIFO is not empty.

```

// Read Enable
assign re = out_valid & out_ready;

// Empty
assign empty = count == 0;

// Out Valid
assign out_valid = !empty;

// Read pointer
always_ff@ ( posedge clk ) begin
    if (!resetn) begin
        rptr <= '0;
    end else begin
        if ( re )
            rptr <= f_incr_ptr ( rptr, SIZE );
    end
end
end

```

4.2.4 Control and Status Logic

We covered FULL, EMPTY and COUNT already. We only need ALMOST_FULL and ALMOST_EMPTY.

If we have given parameters for the count value for almost full and empty then we can just use this. By default, AMOST_FULL_SIZE is the same as SIZE and ALMOST_EMPTY_SIZE is same as 0.

```

assign alm_full = count >= ALMOST_FULL_SIZE;
assign alm_empty = count <= ALMOST_EMPTY_SIZE;

```

4.3 Ring Buffer or Queue

Ring buffer is basically a queue. It is circular in that the read/write pointers wrap around. In strict definition, one should be able to overwrite old data in a ring-buffer but in our case we will handle empty/full conditions. Following is the contract for a ring buffer.

1. An empty queue cannot be read from. That is, the output valid should be low if queue is empty.
2. If a queue is full, then there should be back-pressure and new data cannot be written in.